39Liu, X. and Ji, K. and Fu, Y. and Tam, W. L. and Du, Z. and Yang, Z. and Tang, J., 2022 512Neupane, S. and Ables, M. and Chen, W. and others, 2022 31Liu, F. and Behrooz, F. and others, 2025 321Wei, J. and Wang, X. and Schuurmans, D. and others, 2022 28Chandola, V. and Banerjee, A. and Kumar, V., 2009 212Buczak, A. L. and Guven, E., 2016 11McKeown, N. and Anderson, T. and Balakrishnan, H. and Parulkar, G. and Peterson, L. and Rexford, J. and Shenker, S. and Turner, J., 2008 12Gude, N. and Koponen, T. and Pettit, J. and Pfaff, B. and McKeown, N. and Shenker, S. and Casado, M., 2008 116NTT Laboratories, 2014 114Medved, J. and Varga, R. and Tkacik, A. and Gray, K., 2014 115Berde, P. and Gerola, M. and Hart, J. and Higuchi, Y. and Kobayashi, M. and Koide, T. and others, 2014 122Porras, P. and Shin, S. and Yegneswaran, V. and Fong, M. and Tyson, M. and Gu, G., 2012 123Li, W. and Meng, W. and Kwok, L.-F., 2016 124Bhuiyan, Z. A. and Islam, S. and Islam, M. M. and Ullah, A. A. and Naz, F. and Rahman, M. S., 2023 14Lantz, B. and Heller, B. and McKeown, N., 2010 15Pfaff, B. and Pettit, J. and Koponen, T. and Amidon, K. and Casado, M. and Shenker, S., 2015 110Xia, W. and Wen, Y. and Foh, C. H. and Niyato, D. and Xie, H., 2015 127Chiosi, M. and Clarke, D. and Hampson, P. and others, 2012 128ETSI NFV ISG, 2013 129ETSI NFV ISG, 2014 130Mijumbi, R. and Serrat, J. and Gorricho, J.-L. and Bouten, N. and De Turck, F. and Boutaba, R., 2015 131Mijumbi, R. and Serrat, J. and Gorricho, J.-L. and Bouten, N. and De Turck, F. and Boutaba, R., 2016 132Laghrissi, A. and Taleb, T., 2019 133Leivadeas, A. and Falkner, M., 2022 134Morales, F. and Ruiz, M. and Contreras, L. M. and Ferraris, G. and Stavdas, A. and Cugini, F. and Velasco, L., 2017 135Panayiotou, T. and others, 2022 126Latif, Z. and Sharif, K. and Li, F. and Karim, M. M. and Belghiti, M. A. and Sangeen, A., 2022 22Roberts, S. W., 1959 21Brutlag, J. D., 2000 25Moayedi, H. Z. and Amini, M., 2008 27Fondo-Ferreiro, P. and others, 2026 213Cortes, C. and Vapnik, V., 1995 210Bhuyan, M. H. and Bhattacharyya, D. K. and Kalita, J. K., 2014 217Caruana, R. and Niculescu-Mizil, A., 2006 231Cannady, J. D., 1998 29Ahmed, M. and Mahmood, A. N. and Hu, J., 2016 222Hodo, E. and Bellekens, X. and Hamilton, A. and Dubouil, C. and Pham, Q. N., 2017 224Alduailij, M. and Khan, Q. W. and Tahir, M. and Sardarez, M. and Malik, F., 2022 225Negandhi, P. and Trivedi, Y., 2016 227Shehab, R. and others, 2024 229Staniford-Chen, S. and Cheung, S. and Crawford, R. and Dilger, M. and Frank, J. and Hoagland, J. and Levitt, K. and Weintraub, C. and Yip, D. and Zerkle, D., 1996 226Willems, D. and Kohls, K. and van der Kamp, B. and Vranken, H., 2023 232Lakhina, A. and Crovella, M. and Diot, C., 2004 32Liu, C. and Xie, X. and Zhang, X. and Cui, Y., 2024 33Boateng, G. O. and Sami, H. and Alagha, A. and others, 2024 34Tang, J. and others, 2024 35Huang, Y. and others, 2025 36Hong, J. and others, 2025 37Zhang, L. and Jia, T. and Jia, M. and Wu, Y. and others, 2025 310Touvron, H. and Lavril, T. and Izacard, G. and others, 2023 311Touvron, H. and Martin, L. and Stone, K. and others, 2023 312Jiang, A. Q. and Sablayrolles, A. and Mensch, A. and others, 2023 313Almazrouei, E. and Alobeidli, H. and Alshamsi, A. and others, 2023 314Bai, J. and Bai, S. and others, 2023 315Yang, A. and Yang, B. and others, 2024 316BigScience Workshop, 2022 317Gemma Team, 2024 41Hu, E. J. and Shen, Y. and Wallis, P. and others, 2021 319Vaswani, A. and Shazeer, N. and Parmar, N. and others, 2017 320Ouyang, L. and Wu, J. and Jiang, X. and others, 2022 325SDN-Xplain, 2026 327Lightweight Fine-Tuning of LLMs for Explainable Intrusion Detec-

B. and Diaz-Rodriguez, N. and Del Ser, J. and Bennetot, A. and Tabik, S. and others, 2020 531Kadavath, S. and Conerly, T. and Askell, A. and others, 2022 517Sarker, I. H., 2024 51Danilevsky, M. and Qian, K. and Aharonov, R. and Katsis, Y. and Kawas, B. and Sen, P., 2020 53Ribeiro, M. T. and Singh, S. and Guestrin, C., 2016 54Lundberg, S. M. and Lee, S.-I., 2017 511Ali, T. and Kostakos, P., 2023 520Houssel, N. and Singh, S., 2024 521Pizarro, I. and others, 2025 522Lightweight Fine-Tuning of LLMs for Explainable Intrusion Detection in SDN, 2025 532Guo, C. and Pleiss, G. and Sun, Y. and Weinberger, K. Q., 2017 533Lin, S. and Hilton, J. and Evans, O., 2022 534Zhu, C. and Xu, B. and Wang, Q. and Zhang, Y. and Mao, Z., 2023 61Moustafa, N. and Slay, J., 2015 62Moustafa, N. and Slay, J., 2016 63Sharafaldin, I. and Lashkari, A. H. and Ghorbani, A. A., 2018 64Canadian Institute for Cybersecurity (CIC) and Communications Security Establishment (CSE), 2018 65Ring, M. and Wunderlich, S. and Scheuring, D. and Landes, D. and Hotho, A., 2019 66Tavallaee, M. and Bagheri, E. and Lu, W. and Ghorbani, A. A., 2009 69Lantz, B. and Heller, B. and McKeown, N., 2010 629Mohammadi, S. and Buzura, S. and others, 2023 610Elsayed, M. S. and Le-Khac, N.-A. and Dev, S. and Jurcut, A. D., 2020 614Zhang, Y. and Liu, Q., 2022 615Aceto, G. and Ciuonzo, D. and Montieri, A. and Persico, V. and Pescapa, A., 2024 623Kang, F. and others, 2025 616Tsai, C.-P. and Teng, G. and Wallis, P. and Ding, W., 2025 617Fang, X. and Xu, W. and Tan, Z. and Liu, J. and Huang, Q. and others, 2024 618Li, A. and Wang, S. and Wang, G. and Zhong, S., 2024 630Synthetic Network Traffic Data Generation: A Comparative Analysis, 2024 431Sui, Y. and Zhou, M. and Zhou, M. and Han, S. and Zhang, D., 2024 432Hegselmann, S. and Buendia, A. and Lang, H. and Agrawal, M. and Jiang, X. and Sontag, D., 2023 537Wei, J. and Wang, X. and Schuurmans, D. and others, 2022 414Ding, N. and Qin, Y. and Yang, G. and Wei, F. and others, 2022

Detecção e Mitigação Inteligente de Anomalias em Redes Definidas por Software com Grandes Modelos de Linguagem Open Source: correlação entre o Projeto de Pesquisa e o Sistema Implementado (NOC Lite)correlação entre o Projeto de Pesquisa e o Sistema Implementado (NOC Lite)

Correlação entre o Projeto de Pesquisa e o NOC Lite

Artigo técnico-científico — revisão integrativa

Julho de 2026

Abstract

A detecção e a mitigação inteligente de anomalias em redes definidas por software têm sido reconfiguradas pela adoção de grandes modelos de linguagem *open source*, ajustados por técnicas de fine-tuning eficiente em parâmetros, como LoRA e QLoRA, em substituição a modelos fechados e a abordagens puramente reativas. O presente trabalho correlaciona o projeto de pesquisa que propõe esse paradigma – voltado a anomalias de desempenho em SDN, com reconfiguração de topologia virtual em ciclo fechado – com o sistema efetivamente implementado, denominado *NOC Lite*. A partir de uma revisão da literatura estruturada em seis eixos (SDN/NFV, anomalias por aprendizado de máquina tradicional, LLMs *open source* para NetOps, PEFT/LoRA/QLoRA, inteligência artificial explicável e geração de cenários anômalos com datasets públicos), descreve-se a arquitetura e o pipeline de classificação do sistema construído, composto por três classificadores (heurístico, árvore de decisão e IA generativa via Google Gemini). Em seguida, apresenta-se o mapeamento artigo×sistema e a taxonomia

de gaps estruturais, destacando-se a ausência de SDN, o uso de modelo fechado em vez de LLM *open source*, a inexistência de ajuste fino LoRA/QLoRA, a falta de mitigação em ciclo fechado e a dependência exclusiva de dados sintéticos. Discutem-se as convergências preservadas (baselines, tokenização, justificativa textual, métricas de latência) e propõem-se diretrizes de arquitetura para o alinhamento futuro, incluindo um ciclo fechado com LLM local, LoRA e módulo de mitigação. Conclui-se que o *NOC Lite* realiza parcela da intenção do projeto, mas exige evolução arquitetural substantiva nos eixos centrais para aproximar-se da proposta norteadora.

Palavras-chave: Redes Definidas por Software; Anomalias de Rede; Grandes Modelos de Linguagem; PEFT/LoRA; Inteligência Artificial Explicável.

---

1 Introdução

Redes de computadores modernas operam sob demandas cada vez mais rigorosas de desempenho, confiabilidade e segurança, em um cenário marcado pela heterogeneidade de aplicações e pela complexidade crescente da infraestrutura distribuída. Nesse contexto, a identificação e a classificação de fenômenos anômalos – congestionamento transiente, falhas de enlace, degradação de desempenho – tornam-se atividades fundamentais para a operação eficiente de sistemas computacionais, uma vez que a degradação silenciosa de métricas de rede frequentemente antecede falhas operacionais de maior impacto Rodrigues, Joao Pedro Santos, 2026. Tradicionalmente, a análise de anomalias depende de intervenção manual e de conhecimento especializado, o que a torna propensa a erros e com baixa agilidade de adaptação em tempo real . Modelos baseados em assinaturas ou regras pré-definidas são frequentemente reativos e possuem capacidade limitada para detectar ameaças desconhecidas ou comportamentos de desvio sutil, especialmente quando a fronteira entre operação normal e anômala é difusa .

Nos últimos anos, grandes modelos de linguagem demonstraram capacidades de interpretação, raciocínio contextual e generalização a partir de descrições textuais e dados estruturados . Diferentemente de classificadores tradicionais, tais modelos conseguem combinar múltiplas informações, lidar com incertezas e produzir não apenas uma decisão, mas também uma justificativa textual compatível com a análise humana . O paradigma das redes definidas por software, ao separar o plano de controle do plano de dados, fornece a agilidade necessária para que agentes de inteligência artificial percebam o estado da rede e executem ações de otimização ou segurança de

forma autônoma . Essa convergência – controle programável por software e raciocínio generativo – constitui a motivação central do projeto de pesquisa norteador deste estudo.

O projeto de pesquisa propõe desenvolver um sistema autônomo baseado em LLMs *open source* (ex.: Llama, Falcon) e em ajuste fino eficiente de parâmetros (PEFT), notadamente LoRA/QLoRA, para detectar anomalias de performance em SDN e mitigá-las via reconfiguração proativa de topologia virtual (VNT) e roteamento, em ciclo fechado Rodrigues, Joao Pedro Santos, 2026. O projeto exige, ainda, a utilização de datasets públicos consagrados para validação cruzada, a conversão de atributos numéricos em representações textuais e a entrega de justificativas textuais interpretáveis. Contudo, o sistema implementado – o *NOC Lite* – monitora *hosts* genéricos (endereços IP/DNS) por sondas ICMP/TCP/HTTP/DNS/SNMP, emprega o modelo fechado Google Gemini como classificador generativo e não realiza mitigação nem reconfiguração de rede. Há, portanto, uma distância estrutural entre a proposta do projeto e o sistema construído.

Essa distância motiva a presente análise, que se justifica pela escassez de estudos que confrontem, com evidência de código, uma proposta de pesquisa de LLMs *open source* para SDN e sua implementação concreta. Os objetivos deste trabalho são: expor o referencial teórico das seis áreas de conhecimento envolvidas; descrever o sistema implementado e seu pipeline de classificação; apresentar o mapeamento projeto×sistema e a taxonomia de gaps; discutir convergências, limitações e ameaças à validade; e propor diretrizes de arquitetura para o alinhamento futuro. A contribuição deste artigo é a caracterização sistemática e fundamentada em evidência de código da lacuna entre uma proposta de pesquisa de LLMs *open source* para SDN e sua implementação, servindo tanto como diagnóstico quanto como roteiro de evolução.

2 Referencial Teórico e Revisão da Literatura

A fundamentação deste trabalho articula seis eixos de conhecimento, sintetizados a partir de revisão de literatura estruturada, com as referências citadas validadas nos relatórios de estado da arte do próprio projeto. A seleção do corpus seguiu um procedimento de revisão integrativa , partindo de seis áreas de conhecimento mapeadas a partir do projeto de pesquisa norteador Rodrigues, Joao Pedro Santos, 2026 e de relatórios de análise de código do sistema *NOC Lite*: (1) SDN/NFV; (2) detecção e classificação de anomalias por aprendizado de máquina tradicional; (3) LLMs *open source* e modelos de linguagem para NetOps; (4) fine-tuning eficiente de LLMs (PEFT/LoRA/QLoRA); (5) inteligência artificial explicável (XAI); e

(6) geração de cenários anômalos e datasets públicos. Como critérios de inclusão, admitiram-se obras primárias (artigos, preprints e relatórios técnicos) diretamente relacionadas a um dos seis eixos e com relevância para a análise arquitetural realizada; excluíram-se trabalhos fora do escopo de detecção/mitigação de anomalias em redes e fontes sem validação verificável. O corpus final compreende as fontes efetivamente citadas, distribuídas pelos seis eixos, consolidadas em uma única lista de referências ordenada alfabeticamente conforme a ABNT NBR 6023.

2.1 Redes Definidas por Software e Virtualização de Funções de Rede

O paradigma SDN desacopla o plano de controle do plano de dados por meio de um protocolo southbound padronizado (OpenFlow) e de controladores programáveis (NOX , POX, Ryu , OpenDaylight , ONOS). Essa centralização lógica cria uma única fonte de verdade e uma superfície de programação que, simultaneamente, facilita a automação e introduz novas superfícies de ataque no plano de controle . A emulação (Mininet , Open vSwitch) viabiliza a geração controlada de cenários anômalos . Surveys definitivos consolidam a taxonomia e os desafios de segurança da SDN , fornecendo o vocabulário exato – tabelas de fluxo, contadores, regras – que um agente de inteligência artificial deve compreender para atuar sobre a rede.

Complementarmente, a Virtualização de Funções de Rede (NFV), institucionalizada pelos white papers e pelo framework arquitetural da ETSI , e revisitada por Mijumbi et al. e Laghrissi & Taleb , estende o paradigma ao ciclo de vida de funções virtualizadas e sua orquestração (MANO). Sobre esse alicerce, o Intent-Based Networking (IBN) emerge como a camada de gestão que traduz objetivos de alto nível em configurações concretas e as garante por meio de laços fechados . Trabalhos como Morales et al. e a survey de Panayiotou et al. antecipam a reconfiguração proativa da topologia virtual orientada a dados e a aprendizado de máquina. A detecção de anomalias em SDN transita, assim, de métodos clássicos de ML para a integração de LLMs *open source* ajustados por LoRA/QLoRA, exatamente o movimento defendido pelo projeto norteador.

2.2 Detecção e Classificação de Anomalias por Métodos Tradicionais

A literatura consolida que métodos estatísticos (cartas EWMA , Holt-Winters , ARIMA/SARIMA) e classificadores de ML clássico (SVM , Ran-

dom Forest , árvores de decisão , redes rasas) formam o arcabouço de baselines sobre o qual a detecção moderna é avaliada. Surveys consagrados reforçam que heurísticas de limiar e modelos estatísticos são insubstituíveis como referência de comparação por serem interpretáveis e leves, propriedade particularmente valiosa para operadores de NOC. Para os tipos de anomalia do sistema, há corpus dedicado e real: o DDoS é coberto por baselines de árvore/Random Forest ; a varredura de portas possui precedentes heurísticos e de ML ; a exfiltração é abordada sobre metadados de fluxo ; e o congestionamento ou a falha de enlace encontra raiz em detecção estatística de séries temporais . Tais baselines não são meros artefatos históricos: constituem o termômetro obrigatório contra o qual qualquer abordagem nova, incluindo LLMs, deve ser medida em termos de acurácia, interpretabilidade e custo operacional.

2.3 LLMs Open Source e Modelos de Linguagem para NetOps

Surveys dedicados mapeiam a aplicação de LLMs à operação e gestão de redes (NO&M/NetOps) , estabelecendo o arcabouço conceitual – percepção, raciocínio, ação – sobre o qual se inscreve a classificação generativa de anomalias. O ecossistema de modelos *open source* – LLaMA/Llama 2 , Mistral 7B , Falcon , Qwen/Qwen2.5 , BLOOM e, sobretudo, Gemma (derivado da tecnologia Gemini) – oferece pesos e licenças que tornam viável a substituição de modelos fechados por alternativas locais e ajustáveis . A arquitetura Transformer , o alinhamento por RLHF e o Chain-of-Thought (CoT) fundamentam, respectivamente, a capacidade de raciocínio, de seguir instruções e de produzir justificativa textual. Aplicações concretas incluem detecção de anomalias em SDN , IBN orientado a LLM e agentes de rede autônomos , demonstrando que o paradigma já ultrapassou a fase puramente conceitual.

2.4 Fine-Tuning Eficiente de LLMs (PEFT): LoRA, QLoRA

O ajuste fino eficiente (PEFT) – liderado por LoRA e estendido por QLoRA , AdaLoRA e DoRA – é a via consagrada para adaptar LLMs *open source* a domínios específicos com orçamento de hardware modesto, reduzindo em ordens de magnitude os parâmetros treináveis sem overhead de inferência . Frameworks como LLM-Adapters e LLaMA-Adapter demonstram que LLMs 7B ajustados por PEFT igualam ou superam modelos 175B em zero-shot. No domínio de tráfego de rede, NetGPT , TrafficGPT e TrafficLLM mostram que LLMs podem ser ajustados, com tokenização de domínio, para entender e classificar tráfego. A base few-shot/zero-shot

(Brown , Wei , Achiam) legitima o uso de LLMs sem ajuste de pesos, mas a literatura documenta que o PEFT tipicamente supera o in-context learning em custo e acurácia , o que justifica a exigência de LoRA/QLoRA no projeto norteador.

2.5 Inteligência Artificial Explicável (XAI)

A XAI é requisito fundamental para a adoção confiável de detectores em domínios de alta responsabilidade . Em NLP, o survey de Danilevsky et al. sistematiza como modelos de linguagem produzem explicações; Ribeiro et al. (LIME) e Lundberg & Lee (SHAP) oferecem atribuição local. O protótipo HuntGPT integra um detector de anomalias por ML a um LLM que entrega justificativas textuais, referência explícita do projeto. Estudos recentes exploram LLMs para detecção de intrusão explicável e explicabilidade em SDN . A calibração e a confiança das justificativas são temas ativos , pois explicações de LLM podem ser plausíveis e incorretas, exigindo verificação cruzada com atribuição numérica de features.

2.6 Geração de Cenários Anômalos e Datasets Públicos

A literatura estabelece UNSW-NB15 e CSE-CIC-IDS2018 como benchmarks modernos, superando os obsoletos KDD99/NSL-KDD . A geração dessas bases segue perfis comportamentais e emulação (Mininet, GNS3, PNET-Lab) . Datasets SDN específicos (InSDN , SNT, DDoS2019) e técnicas de data augmentation (GAN/VAE/SMOTE) endereçam o desbalanceamento de classes . A conversão de atributos numéricos em representações textuais – formalizada por AnoLLM e revisada em surveys de LLMs em dados tabulares – abre caminho para detectores baseados em LLMs, eixo conectado diretamente à inferência do sistema analisado . A ausência de validação cruzada contra tais bases é, precisamente, um dos pontos de tensão entre o projeto e o sistema implementado.

3 O Sistema Implementado: NOC Lite

O *NOC Lite* é um sistema de monitoramento de disponibilidade e performance de *hosts* genéricos, estruturado como um monolito modular em FastAPI com persistência em SQLite e um front-end estático. A arquitetura de contêineres cobre o front-end (HTML/JS), a API backend (FastAPI/Uvicorn), o motor de monitoramento (`asyncio`), o classificador de anomalias, o simulador, o notificador (Telegram) e o banco de dados SQLite. Ponto crítico de

arquitetura: o classificador de anomalias **não participa do laço de monitoramento** – é acionado sob demanda via `GET /api/anomalies/classify`, jamais pelo *MonitorEngine*, operando como ferramenta de diagnóstico e não como sensor integrado ao pipeline de detecção.

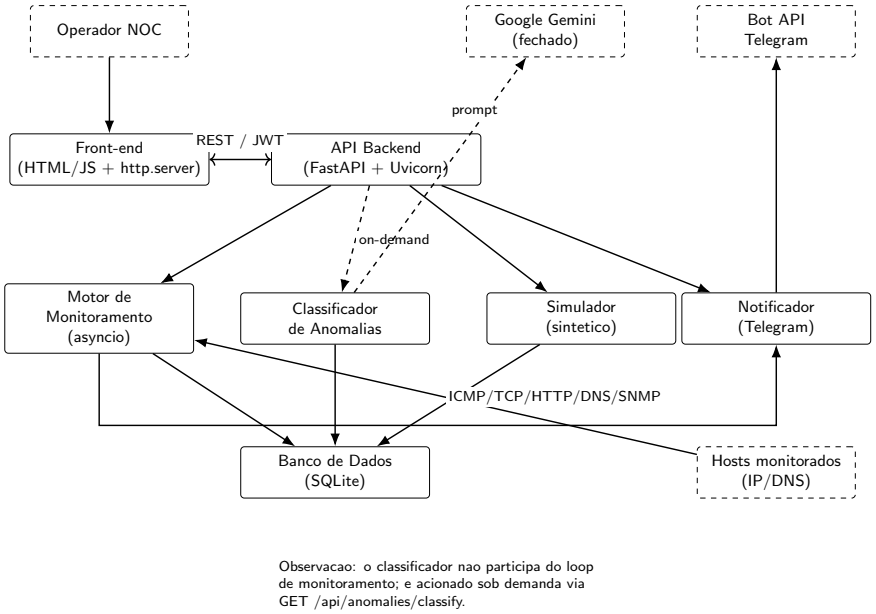


Figure 1: Arquitetura de contêineres do NOC Lite (C4, nível contêiner). Fonte: elaborada a partir de `arquitetura_sistema.md` (Dan).

3.1 Pipeline de Classificação

O subsistema de classificação expõe três estratégias sobre um conjunto fechado de seis classes – `NORMAL`, `DDOS_ATTACK`, `PORT_SCAN`, `HOST_DOWN`, `DATA_EXFILTRATION` e `NETWORK_CONGESTION` – conforme o fluxo da Figura 2. A primeira estratégia, o classificador heurístico (`classify_heuristic`), aplica regras de limiar estático sobre latência, CPU, memória e taxas de sucesso de sondas; exemplos típicos são `network_in_bps > 50 Mb/s` e `cpu_usage > 75` que levam a `DDOS_ATTACK`, e `ping_latency > 120 ms` que conduz a `NETWORK_CONGESTION`, produzindo confiança fixa e explicação textual. A segunda, o classificador de aprendizado de máquina (`classify_ml`), utiliza um `DecisionTreeClassifier(max_depth=5)` do scikit-learn treinado **exclusivamente com dados sintéticos** (1.200 amostras, 200 por classe), sem PEFT e sem LLM, comportando-se, na prática, como uma "árvore-cópia" da heurística. A terceira, a IA generativa (`classify_generative_ai`), monta um prompt estruturado com o snapshot tokenizado e o envia ao modelo fechado `gemini-1.5-flash`, retornando JSON com `classification`,

`confidence` e `explanation`, com fallback para a heurística em caso de falha.

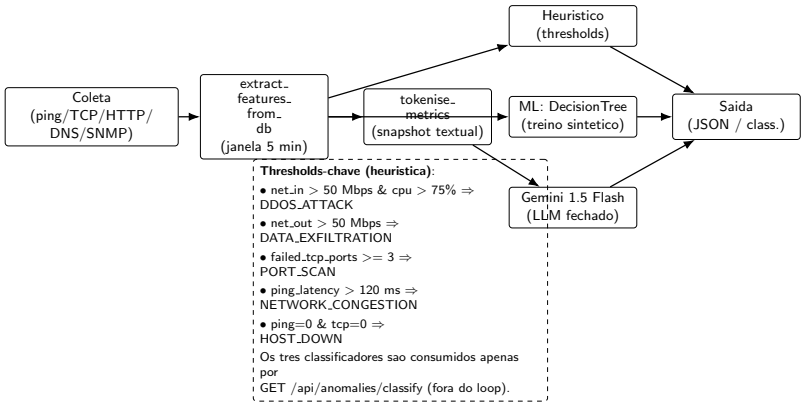


Figure 2: Pipeline de classificação de anomalias do NOC Lite: extração de features (janela de 5 min), três classificadores paralelos e tokenização para o LLM. Fonte: `notas_coder.md` (John).

A função `tokenise_metrics` converte o vetor numérico de dez features em texto estruturado (ex.: `PingSuccessRate: 0.95 | AvgPingLatency: 12.3 ms | CPUUsage: 34.1%`), espelhando a recomendação do projeto de converter atributos numéricos em representações textuais . Esse texto é a ponte que permite ao LLM "ler" os limiares de rede em linguagem natural, materializando o eixo de serialização tabular→texto discutido na fundamentação.

3.2 Simulador

O `anomaly_simulator.py` gera cenários anômalos sintéticos (`generate_simulated_metrics`) para as seis classes via `random.uniform`, e injeta esses vetores diretamente no banco (`run_simulated _check_cycle`), podendo popular histórico retroativo. O agendamento em tempo real usa estado em memória. Trata-se de dados 100% sintéticos, sem dependência de datasets públicos, o que confere privacidade e reprodutibilidade, mas inviabiliza a comparação com o estado da arte – tensionando diretamente o requisito de validação cruzada do projeto.

4 Mapeamento Projeto × Sistema e Gaps (Resultados)

A Tabela 1 sintetiza a correspondência entre os objetivos/componentes do projeto norteador Rodrigues, Joao Pedro Santos, 2026 e o que o NOC Lite efetivamente entrega, com o respectivo status de alinhamento. A Figura 3 ilustra essa relação de forma visual, evidenciando que as convergências concentram-se nas margens do sistema (baselines, tokenização, justificativa, métricas), ao passo que os eixos centrais da proposta (SDN, LLM aberto, LoRA, mitigação, datasets) permanecem fora do alcance implementado.

Table 1: Mapeamento Projeto de Pesquisa × NOC Lite.

#	**Objetivo/Componente do Projeto**	**Entrega do NOC Lite**	**Status**
1	LLM *open source* (Llama/Falcon)	Gemini 1.5 Flash (fechado); Árvore de Decisão	Divergente
2	Fine-tuning PEFT (LoRA/QLoRA)	Inexistente; ML é árvore treinada do zero	Ausente
3	SDN virtualizada (Mininet/OVS)	Hosts IP genéricos; ICMP/TCP/HTTP/DNS/SNMP	Ausente
4	Mitigação via reconfiguração VNT/roteamento	Apenas detecção + alerta Telegram; sem ação corretiva	Ausente
5	Anomalias de performance (congestionamento/falha)	Mistura anomalias de segurança com congest. genérico	Parcial
6	Datasets públicos (UNSW-NB15/CIC-IDS2018)	Dados 100% sintéticos (`random.uniform`)	Divergente
7	Baselines heurística/ML	Heurística + DecisionTree	Convergente
8	Tokenização de métricas em texto estruturado	`tokenise_metrics`	Convergente
9	Justificativa textual (XAI)	`explanation` (com modelo fechado)	Convergente

#	Objetivo/Componente do Projeto	Entrega do NOC Lite	Status
10	Avaliação de tempo de resposta por classificador	`latency_ms` por classificador	Convergente
11	Métricas resumidas (throughput/latência/jitter/perda)	SLA/jitter/tendência/health score + SNMP	Convergente
12	Ciclo fechado coleta→classificação→mitigação	Loop aberto: coleta→severidade→alerta	Aberto

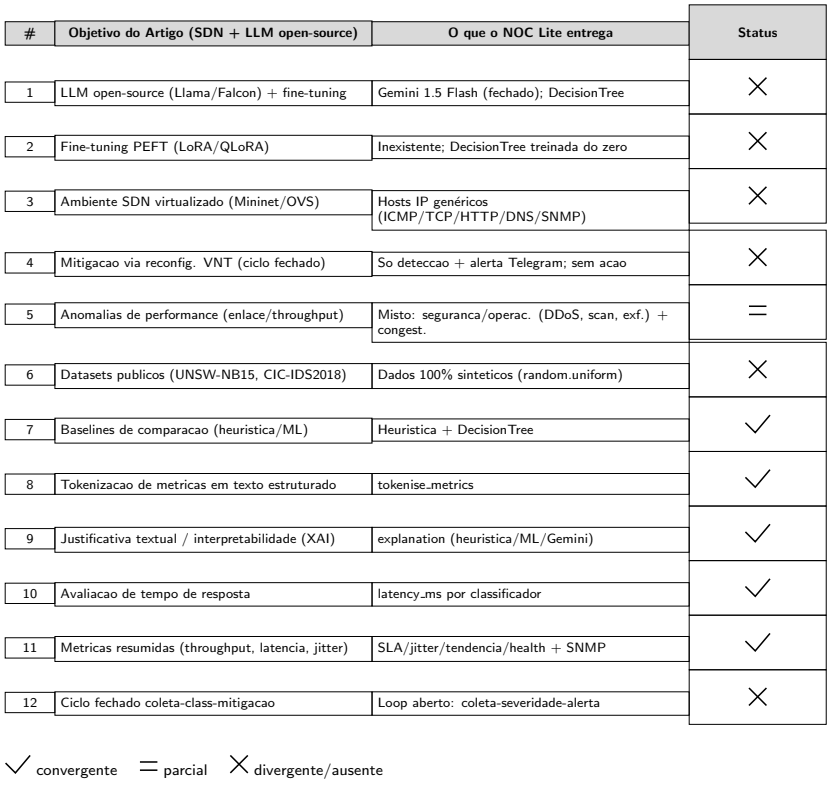


Figure 3: Mapeamento visual entre os objetivos do artigo norteador e o sistema NOC Lite. Fonte: elaborada por Chris.

4.1 Taxonomia de Gaps

A análise arquitetural identifica sete gaps, sendo seis de severidade crítica ou maior, conforme a Figura 4. Cada gap é sustentado por evidência de

código, conforme a Tabela 2.

Table 2: Taxonomia de gaps (NOC Lite × Projeto).

ID	Gap	Evidência no código / Descrição	Severidade
GAP-1	SDN vs. hosts genéricos	Busca por `sdn|mininet|vnt|controller` → 0 ocorrências; `Host.address` arbitrário	Crítico
GAP-2	Gemini fechado vs. LLM *open source*	`genai.GenerativeModel("gemini-1.5-flash");` depende de `GEMINI_API_KEY`	Crítico
GAP-3	Ausência de LoRA/QLoRA	Busca por `lora|qlora|peft|fine.?tun` → 0; único modelo treinável é DecisionTree	Crítico
GAP-4	Ausência de mitigação / loop aberto	`scheduler.py` não referencia nenhum `classify_*`; inferência só em `/api/anomalies/classify`	Crítico
GAP-5	Dados sintéticos vs. datasets públicos	`random.uniform` em `anomaly_simulator.py`; busca `unsw|cic-ids` → 0	Maior
GAP-6	Performance vs. segurança/operacionais	Classes majoritariamente de segurança; congest. por limiar genérico de latência de ping	Maior
GAP-7	Interpretabilidade com modelo fechado	`explanation` provém do Gemini; sem rastreabilidade de pesos nem reprodutibilidade local	Menor

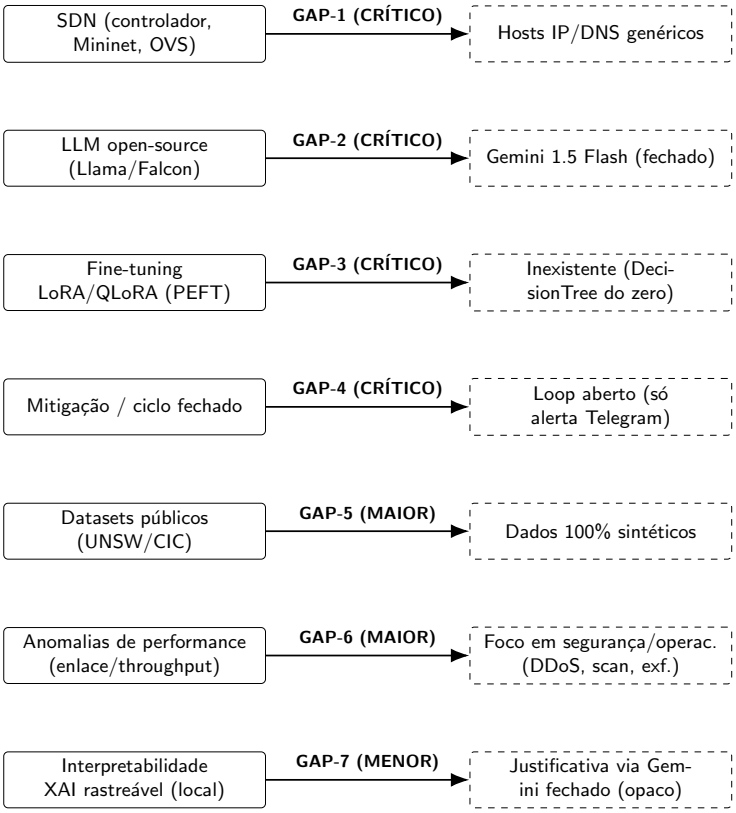


Figure 4: Taxonomia de gaps estruturais entre o projeto norteador e o NOC Lite. Fonte: elaborada por Chris.

O GAP-1 caracteriza a ausência de SDN: o projeto fundamenta toda a proposta no paradigma SDN, enquanto o NOC Lite monitora hosts IP/DNS arbitrários, sem controlador, topologia virtualizada ou planos separados . O GAP-2 diz respeito ao modelo fechado: o projeto exige LLMs *open source* e classifica explicitamente o Gemini como modelo fechado Rodrigues, Joao Pedro Santos, 2026; o sistema, porém, o utiliza em regime zero-shot . O GAP-3 revela a ausência de LoRA/QLoRA: a seção de PEFT do projeto não tem correspondente, e o único modelo treinável é uma árvore de decisão do zero . O GAP-4 aponta o loop aberto: a classificação por IA não está integrada ao laço de detecção, de modo que o ciclo observa e alerta, sem atuação autônoma . O GAP-5 remete aos dados sintéticos: o treino e o few-shot repousam no simulador, ausente a validação cruzada com UNSW-NB15 . O GAP-6 diz respeito ao escopo: o foco do projeto é performance (congestionamento, falha de enlace, degradação de desempenho), ao passo que o sistema privilegia classes de segurança . Por fim, o GAP-7 associa a interpretabilidade a modelo fechado: a justificativa provém de modelo opaco, fragilizando a promessa de XAI do projeto .

5 Discussão

5.1 Convergências preservadas

Apesar dos desalinhamentos estruturais, o NOC Lite honra parcela da intenção do projeto em pontos que não devem ser subestimados. O primeiro ponto é a presença de baselines de comparação: o sistema implementa exatamente a heurística e a árvore de decisão exigidas, permitindo comparação direta lado a lado , o que materializa o objetivo específico de avaliar o desempenho da abordagem generativa contra referências clássicas. O segundo ponto é a tokenização de métricas: `tokenise_metrics` converte o vetor numérico em representação semiestruturada, alinhada à recomendação de conversão textual , antecipando a ponte de entrada que um LLM ajustado por LoRA consumiria. O terceiro ponto é a justificativa textual ou XAI: todos os classificadores retornam `explanation`, atendendo ao objetivo qualitativo de interpretabilidade , ainda que com reserva quanto à rastreabilidade do modelo. O quarto ponto é a medição de tempo de resposta: o endpoint mede `latency_ms` de cada classificador, entregando a métrica de desempenho exigida . O quinto ponto é a captura de métricas resumidas de rede: SLA, jitter, tendência e health-score capturam latência, perda e jitter, coincidentes com as métricas de performance do projeto . Esses cinco pontos formam o alicerce sobre o qual a evolução proposta deve ser construída.

5.2 Limitações e ameaças à validade

A limitação mais severa é o viés de dados sintéticos: o classificador de ML é treinado 100% com o próprio simulador, que compartilha os mesmos limiares da heurística; a árvore aprende essencialmente a copiar a heurística em forma de árvore, com alto desempenho no sintético e provável colapso em produção . A confiança não é calibrada: a do Gemini é autorreportada e a do ML é probabilidade de folha não calibrada , tornando a comparação entre classificadores metodologicamente assimétrica. A orquestração está ausente, pois a classificação encontra-se fora do ciclo de monitoramento, impossibilitando detecção contínua. Na validade externa, a ausência de datasets públicos impede a comparação com o estado da arte .

As ameaças à validade interna incluem a dependência de API externa paga (Gemini) e a fragilidade do parser JSON, que checa apenas a presença da chave `classification`. As ameaças à validade de construção dizem respeito aos limiares fixos de 50 Mb/s e 120 ms, universais e sem baseline por host/link, gerando falsos positivos em enlaces de capacidades distintas . Tais fragilidades não invalidam o sistema como protótipo de diagnóstico, mas restringem sua pretensão de cumprir a proposta norteadora em sua plenitude.

6 Diretrizes de Arquitetura para Alinhamento

Para reduzir a distância entre o NOC Lite e o projeto, propõe-se uma evolução arquitetural incremental, respeitando o que já existe e fechando os sete gaps. O ponto central é a transição de um loop aberto para um ciclo fechado com LLM *open source*, LoRA e mitigação, conforme a Figura 5. A sequência de evolução, espelhando o cronograma do projeto, desdobra-se nos seguintes passos. O primeiro passo é a substituição do modelo fechado por um LLM local (fecha GAP-2 e GAP-3): introduzir um contêiner de inferência local (vLLM/llama.cpp/Ollama) servindo Llama 3 (7–13B) ou Falcon via endpoint

compatível com OpenAI, substituir `gemini-1.5-flash` por cliente HTTP local e aplicar LoRA/QLoRA (rank 16–32, alpha 32) sobre o modelo base, reutilizando os prompts de `tokenise_metrics` . O segundo passo é a criação de um pipeline de dados com datasets públicos (fecha GAP-5): módulo `data_ingest` que baixa e normaliza UNSW-NB15, CSE-CIC-IDS2018 e Synthetic SDN para o schema de features, treinando LoRA e a árvore com dados reais, preservando o simulador para demonstração . O terceiro passo é o ambiente SDN e o fechamento do ciclo (fecha GAP-1 e GAP-4): contêiner Mininet + controlador Ryu que emule a topologia e exponha telemetria de enlace/fluxo; generalização de `Host`/`CheckResult` para telemetria de fluxo SDN; criação de `mitigation_engine` que mapeia `classification` em ação corretiva (re-rota, QoS, drop de fluxo) via NorthBound API do controlador; integração do classificador dentro do `MonitorEngine`, fechando o ciclo . O quarto passo é o realinhamento do catálogo de anomalias (fecha GAP-6): separar claramente performance SDN (congelamento de enlace, falha de enlace, degradação de throughput/jitter) de segurança/operacional (DDoS, port scan, exfiltração, host down), priorizando a primeira família . O quinto passo é a preservação dos pontos fortes: manter `tokenise_metrics`, o par baseline heurístico + ML, a medição de `latency_ms` e o formato de `explanation` como base do novo pipeline . O sexto e último passo é o roadmap de alinhamento experimental: contêiner de LLM *open source* local + `tokenise_metrics` em zero-shot; ingestão dos três datasets + treino LoRA/QLoRA; Mininet + controlador + telemetria de fluxo; `mitigation_engine` + integração da classificação no `MonitorEngine`; e fechamento do ciclo com avaliação comparativa (heurística vs. ML vs. LLM *open source* com/sem LoRA), reutilizando `latency_ms` e acrescentando acurácia e consistência.

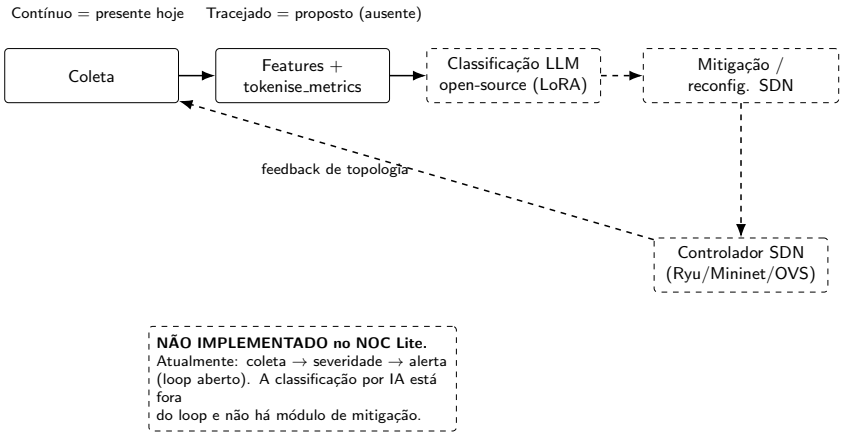


Figure 5: Arquitetura proposta de ciclo fechado: coleta (SDN + hosts) → tokenização → classificação (LLM *open source* + LoRA) → mitigação (módulo de ação) → reconfiguração via controlador. Fonte: elaborada por Chris.

7 Conclusão

O presente trabalho correlacionou o projeto de pesquisa que propõe detecção e mitigação de anomalias em SDN com LLMs *open source* ajustados por PEFT/LoRA Rodrigues, Joao Pedro Santos, 2026 com o sistema efetivamente implementado, o NOC Lite.

Evidenciou-se que o sistema realiza com sucesso a parcela de baselines e de representação textual do projeto – heurística, árvore de decisão, tokenização de métricas, justificativa textual e medição de latência – mas diverge estruturalmente nos eixos centrais da proposta. Especificamente, o NOC Lite monitora hosts genéricos e não SDN; emprega o modelo fechado Gemini em vez de LLM *open source*; não aplica LoRA/QLoRA; não realiza mitigação nem opera em ciclo fechado; e depende exclusivamente de dados sintéticos, sem validação cruzada com datasets públicos.

Tais gaps não são ajustes pontuais: exigem evolução arquitetural substantiva, sintetizada nas diretrizes de um ciclo fechado com LLM local, LoRA e módulo de mitigação. As convergências identificadas funcionam como alicerce seguro sobre o qual construir essa evolução, uma vez que o `tokenise_metrics`, os baselines e a medição de latência já cumprem objetivos específicos do projeto. Como trabalho futuro, recomenda-se a implementação do `mitigation_engine` e a condução do experimento comparativo head-to-head entre LLM *open source* ajustado por QLoRA e LLM fechado zero-shot sobre o mesmo conjunto de métricas de rede – lacuna exatamente apontada pela literatura . A realização desse experimento é o passo necessário para que a implementação deixe de ser um diagnóstico por similaridade e passe a constituir, de fato, a validação empírica da proposta norteadora.

References

Rodrigues, Joao Pedro Santos, 2026. Deteccao e Mitigacao Inteligente de Anomalias em Redes Definidas por Software com Grandes Modelos de Linguagem Open Source.

Liu, X. and Ji, K. and Fu, Y. and Tam, W. L. and Du, Z. and Yang, Z. and Tang, J., 2022. P-Tuning v2: Prompt Tuning Can Be Comparable to Fine-tuning Universally Across Scales and Tasks.

Neupane, S. and Ables, M. and Chen, W. and others, 2022. Explainable Intrusion Detection Systems (X-IDS): A Survey of Current Methods, Challenges, and Opportunities.

Liu, F. and Behrooz, F. and others, 2025. Large Language Models for Computer Networking Operations and Management: A Survey on Applications, Key Techniques, and Opportunities.

Wei, J. and Wang, X. and Schuurmans, D. and others, 2022. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models.

Chandola, V. and Banerjee, A. and Kumar, V., 2009. Anomaly Detection: A Survey.

Buczak, A. L. and Guven, E., 2016. A Survey of Data Mining and Machine Learning Methods for Cyber Security Intrusion Detection.

McKeown, N. and Anderson, T. and Balakrishnan, H. and Parulkar, G. and Peterson, L. and Rexford, J. and Shenker, S. and Turner, J., 2008. OpenFlow: Enabling Innovation in Campus Networks.

Gude, N. and Koponen, T. and Pettit, J. and Pfaff, B. and McKeown, N. and Shenker, S. and Casado, M., 2008. NOX: Towards an Operating System for Networks.

NTT Laboratories, 2014. Ryu SDN Framework – Open-source SDN Platform Software.

Medved, J. and Varga, R. and Tkacik, A. and Gray, K., 2014. OpenDaylight: Towards a Model-Driven SDN Controller Architecture.

Berde, P. and Gerola, M. and Hart, J. and Higuchi, Y. and Kobayashi, M. and Koide, T. and others, 2014. ONOS: Towards an Open, Distributed SDN OS.

Porras, P. and Shin, S. and Yegneswaran, V. and Fong, M. and Tyson, M. and Gu, G., 2012. A Security Enforcement Kernel for OpenFlow Networks.

Li, W. and Meng, W. and Kwok, L.-F., 2016. A Survey on OpenFlow-based Software Defined Networks: Security Challenges and Countermeasures.

Bhuiyan, Z. A. and Islam, S. and Islam, M. M. and Ullah, A. A. and Naz, F. and Rahman, M. S., 2023. On the (In)Security of the Control Plane of SDN Architecture: A Survey.

Lantz, B. and Heller, B. and McKeown, N., 2010. A Network in a Laptop: Rapid Prototyping for Software-Defined Networks.

Pfaff, B. and Pettit, J. and Koponen, T. and Amidon, K. and Casado, M. and Shenker, S., 2015. The Design and Implementation of Open vSwitch.

Xia, W. and Wen, Y. and Foh, C. H. and Niyato, D. and Xie, H., 2015. A Survey on Software-Defined Networking.

Chiosi, M. and Clarke, D. and Hampson, P. and others, 2012. Network Functions Virtualisation (NFV): White Paper #1.

ETSI NFV ISG, 2013. GS NFV 002 V1.1.1: Network Functions Virtualisation (NFV); Architectural Framework.

ETSI NFV ISG, 2014. GS NFV-MAN 001 V1.1.1: Network Functions Virtualisation (NFV); Management and Orchestration.

Mijumbi, R. and Serrat, J. and Gorricho, J.-L. and Bouten, N. and De Turck, F. and Boutaba, R., 2015. Network Function Virtualization: State-of-the-Art and Research Challenges.

Mijumbi, R. and Serrat, J. and Gorricho, J.-L. and Bouten, N. and De Turck, F. and Boutaba, R., 2016. Network Function Virtualization: State-of-the-Art and Research Challenges.

Laghrissi, A. and Taleb, T., 2019. A Survey on the Placement of Virtual Resources and Virtual Network Functions.

Leivadeas, A. and Falkner, M., 2022. A Survey on Intent-Based Networking.

Morales, F. and Ruiz, M. and Contreras, L. M. and Ferraris, G. and Stavdas, A. and Cugini, F. and Velasco, L., 2017. Virtual Network Topology Adaptability Based on Data Analytics for Traffic Prediction.

Panayiotou, T. and others, 2022. Survey on Machine Learning for Traffic-Driven Service and Virtual Network Topology Reconfiguration.

Latif, Z. and Sharif, K. and Li, F. and Karim, M. M. and Belghiti, M. A. and Sangeen, A., 2022. A Machine Learning-Based Anomaly Prediction Service for Software-Defined Networks.

Roberts, S. W., 1959. Control Chart Tests Based on Geometric Moving Averages.

Brutlag, J. D., 2000. Aberrant Behavior Detection in Time Series for Network Monitoring.

Moayedi, H. Z. and Amini, M., 2008. ARIMA Model for Network Traffic Prediction and Anomaly Detection.

Fondo-Ferreiro, P. and others, 2026. Real-time SARIMA-based anomaly detection in operator networks.

Cortes, C. and Vapnik, V., 1995. Support-Vector Networks.

Bhuyan, M. H. and Bhattacharyya, D. K. and Kalita, J. K., 2014. Network Anomaly Detection: Methods, Systems and Tools.

Caruana, R. and Niculescu-Mizil, A., 2006. An Empirical Comparison of Supervised Learning Algorithms.

Cannady, J. D., 1998. Artificial Neural Networks for Misuse Detection.

Ahmed, M. and Mahmood, A. N. and Hu, J., 2016. A Survey of Network Anomaly Detection Techniques.

Hodo, E. and Bellekens, X. and Hamilton, A. and Dubouil, C. and Pham, Q. N., 2017. Threat Analysis of IoT Networks Using Artificial Neural Network Intrusion Detection System.

Alduailij, M. and Khan, Q. W. and Tahir, M. and Sardarez, M. and Malik, F., 2022. Machine-Learning-Based DDoS Attack Detection Using Mutual Information and Random Forest Feature Importance Method.

Negandhi, P. and Trivedi, Y., 2016. Intrusion Detection System Using Random Forest on the NSL-KDD Dataset.

Shehab, R. and others, 2024. Improving Port Scan Cybersecurity Risks Detection using Feature Selection and Machine Learning.

Staniford-Chen, S. and Cheung, S. and Crawford, R. and Dilger, M. and Frank, J. and Hoagland, J. and Levitt, K. and Weintraub, C. and Yip, D. and Zerkle, D., 1996. GrIDS – A Graph-Based Intrusion Detection System for Large Networks.

Willems, D. and Kohls, K. and van der Kamp, B. and Vranken, H., 2023. Data Exfiltration Detection on Network Metadata with Autoencoders.

Lakhina, A. and Crovella, M. and Diot, C., 2004. Diagnosing Network-Wide Traffic Anomalies.

Liu, C. and Xie, X. and Zhang, X. and Cui, Y., 2024. Large Language Models for Networking: Workflow, Advances and Challenges.

Boateng, G. O. and Sami, H. and Alagha, A. and others, 2024. A Survey on Large Language Models for Communication, Network, and Service Management: Application Insights, Challenges, and Future Directions.

Tang, J. and others, 2024. Utilizing Large Language Models for Advanced Optimization and Intelligent Management in Space-Air-Ground Integrated Networks.

Huang, Y. and others, 2025. Large Language Models for Networking: Applications, Enabling Techniques, and Challenges.

Hong, J. and others, 2025. A Comprehensive Survey on LLM-Based Network Management and Orchestration.

Zhang, L. and Jia, T. and Jia, M. and Wu, Y. and others, 2025. A Survey of AIOps in the Era of Large Language Models.

Touvron, H. and Lavril, T. and Izacard, G. and others, 2023. LLaMA: Open and Efficient Foundation Language Models.

Touvron, H. and Martin, L. and Stone, K. and others, 2023. Llama 2: Open Foundation and Fine-Tuned Chat Models.

Jiang, A. Q. and Sablayrolles, A. and Mensch, A. and others, 2023. Mistral 7B.

Almazrouei, E. and Alobeidli, H. and Alshamsi, A. and others, 2023. The Falcon Series of Open Language Models.

Bai, J. and Bai, S. and others, 2023. Qwen Technical Report.

Yang, A. and Yang, B. and others, 2024. Qwen2.5 Technical Report.

BigScience Workshop, 2022. BLOOM: A 176B-Parameter Open-Access Multilingual Language Model.

Gemma Team, 2024. Gemma: Open Models Based on Gemini Research and Technology.

Hu, E. J. and Shen, Y. and Wallis, P. and others, 2021. LoRA: Low-Rank Adaptation of Large Language Models.

Vaswani, A. and Shazeer, N. and Parmar, N. and others, 2017. Attention Is All You Need.

Ouyang, L. and Wu, J. and Jiang, X. and others, 2022. Training Language Models to Follow Instructions with Human Feedback (InstructGPT).

SDN-Xplain, 2026. SDN-Xplain: LLM-Driven Anomaly Detection and Human-Interpretable Explanation in SDN.

Lightweight Fine-Tuning of LLMs for Explainable Intrusion Detection in SDN, 2025. Lightweight Fine-Tuning of LLMs for Explainable Intrusion Detection in SDN.

Hossain, M. K. and others, 2025. NetIntent: Leveraging Large Language Models for End-to-End Intent-Based SDN Automation.

Adanza, D. and others, 2025. Leveraging Generative AI for Intent-Based Networking.

Towards Intent-Based Network Management: Large Language Models for Intent Extraction in 5G Core Networks, 2024. Towards Intent-Based Network Management: Large Language Models for Intent Extraction in 5G Core Networks.

Chen, Y. and others, 2024. NetGPT: An AI-Native Network Architecture for Provisioning Beyond Personalized Generative Services.

Exploring LLM-Based Agents for Root Cause Analysis, 2024. Exploring LLM-Based Agents for Root Cause Analysis.

Empowering AIOps: Leveraging Large Language Models for IT Operations Management, 2025. Empowering AIOps: Leveraging Large Language Models for IT Operations Management.

Wang, Z. and others, 2025. Intent-Driven Network Management with Multi-Agent LLMs.

LLM-Driven Multi-Agent Framework for Autonomous Network Slice Management, 2026. LLM-Driven Multi-Agent Framework for Autonomous Network Slice Management.

Dettmers, T. and Pagnoni, A. and Holtzman, A. and Zettlemoyer, L., 2023. QLoRA: Efficient Finetuning of Quantized LLMs.

Zhang, Q. and Chen, M. and Bukharin, A. and others, 2023. AdaLoRA: Adaptive Budget Allocation for Parameter-Efficient Fine-Tuning.

Liu, S.-Y. and Wang, C.-Y. and Yin, H. and others, 2024. DoRA: Weight-Decomposed Low-Rank Adaptation.

Houlsby, N. and Giurgiu, A. and Jastrzebski, S. and others, 2019. Parameter-Efficient Transfer Learning for NLP.

Han, Z. and Gao, C. and Liu, J. and Zhang, J. and Zhang, S. Q., 2024. Parameter-Efficient Fine-Tuning for Large Models: A Comprehensive Survey.

Hu, Z. and Wang, L. and Lan, Y. and others, 2023. LLM-Adapters: An Adapter Family for Parameter-Efficient Fine-Tuning of Large Language Models.

Zhang, R. and Han, J. and Liu, C. and Gao, P. and others, 2024. LLaMA-Adapter: Efficient Fine-tuning of Language Models with Zero-init Attention.

Meng, X. and Lin, C. and Wang, Y. and Zhang, Y., 2025. NetGPT: Generative Pretrained Transformer for Network Traffic.

Ginige, Y. and Dahanyayaka, T. and Huang, Y. and Jourjon, G. and Seneviratne, S., 2024. TrafficGPT: An LLM Approach for Open-Set Encrypted Traffic Classification.

Cui, T. and Lin, X. and Li, S. and Chen, M. and others, 2025. TrafficLLM: Enhancing Large Language Models for Network Traffic Analysis with Generic Traffic Representation.

Brown, T. B. and Mann, B. and Ryder, N. and others, 2020. Language Models are Few-Shot Learners.

Achiam, J. and Adler, S. and Agarwal, S. and others, 2023. GPT-4 Technical Report.

Arrieta, A. B. and Diaz-Rodriguez, N. and Del Ser, J. and Bennetot, A. and Tabik, S. and others, 2020. Explainable Artificial Intelligence (XAI): Concepts, Taxonomies, Opportunities and Challenges toward Responsible AI.

Kadavath, S. and Conerly, T. and Askell, A. and others, 2022. Language Models (Mostly) Know What They Know.

Sarker, I. H., 2024. Explainable AI for Cybersecurity Automation, Intelligence and Trustworthiness in Digital Twin: Methods, Taxonomy, Challenges and Prospects.

Danilevsky, M. and Qian, K. and Aharonov, R. and Katsis, Y. and Kawas, B. and Sen, P., 2020. A Survey of the State of Explainable AI for Natural Language Processing.

Ribeiro, M. T. and Singh, S. and Guestrin, C., 2016. "Why Should I Trust You?": Explaining the Predictions of Any Classifier.

Lundberg, S. M. and Lee, S.-I., 2017. A Unified Approach to Interpreting Model Predictions.

Ali, T. and Kostakos, P., 2023. HuntGPT: Integrating Machine Learning-Based Anomaly Detection and Explainable AI with Large Language Models (LLMs).

Houssel, N. and Singh, S., 2024. Towards Explainable Network Intrusion Detection using Large Language Models.

Pizarro, I. and others, 2025. X-Flow: Explanation-Augmented Prompting for LLM-Based Intrusion Detection.

Lightweight Fine-Tuning of LLMs for Explainable Intrusion Detection in SDN, 2025. Lightweight Fine-Tuning of LLMs for Explainable Intrusion Detection in SDN.

Guo, C. and Pleiss, G. and Sun, Y. and Weinberger, K. Q., 2017. On Calibration of Modern Neural Networks.

Lin, S. and Hilton, J. and Evans, O., 2022. Teaching Models to Express Their Uncertainty in Words.

Zhu, C. and Xu, B. and Wang, Q. and Zhang, Y. and Mao, Z., 2023. On the Calibration of Large Language Models and Alignment.

Moustafa, N. and Slay, J., 2015. UNSW-NB15: A Comprehensive Data Set for Network Intrusion Detection Systems.

Moustafa, N. and Slay, J., 2016. The Evaluation of Network Anomaly Detection Systems: Statistical Analysis of the UNSW-NB15 Data Set and the Comparison with the KDD99 Data Set.

Sharafaldin, I. and Lashkari, A. H. and Ghorbani, A. A., 2018. Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization.

Canadian Institute for Cybersecurity (CIC) and Communications Security Establishment (CSE), 2018. CSE-CIC-IDS2018 Dataset.

Ring, M. and Wunderlich, S. and Scheuring, D. and Landes, D. and Hotho, A., 2019. A Survey of Network-Based Intrusion Detection Data Sets.

Tavallaee, M. and Bagheri, E. and Lu, W. and Ghorbani, A. A., 2009. A Detailed Analysis of the KDD CUP 99 Data Set.

Lantz, B. and Heller, B. and McKeown, N., 2010. A Network in a Laptop: Rapid Prototyping for Software-Defined Networks.

Mohammadi, S. and Buzura, S. and others, 2023. A Hybrid Software and Hardware SDN Simulation Testbed.

Elsayed, M. S. and Le-Khac, N.-A. and Dev, S. and Jurcut, A. D., 2020. InSDN: A Novel SDN Intrusion Dataset.

Zhang, Y. and Liu, Q., 2022. On IoT Intrusion Detection Based on Data Augmentation for Enhancing Learning on Unbalanced Samples.

Aceto, G. and Ciuonzo, D. and Montieri, A. and Persico, V. and Pescapa, A., 2024. Synthetic and Privacy-Preserving Traffic Trace Generation Using Generative AI Models for Training Network Intrusion Detection Systems.

Kang, F. and others, 2025. A Class Imbalance Data Augmentation Method for Network Intrusion Detection Based on M2M-VAEGAN.

Tsai, C.-P. and Teng, G. and Wallis, P. and Ding, W., 2025. AnoLLM: Large Language Models for Tabular Anomaly Detection.

Fang, X. and Xu, W. and Tan, Z. and Liu, J. and Huang, Q. and others, 2024. Large Language Models on Tabular Data: A Survey.

Li, A. and Wang, S. and Wang, G. and Zhong, S., 2024. Anomaly Detection of Tabular Data Using LLMs.

Synthetic Network Traffic Data Generation: A Comparative Analysis, 2024. Synthetic Network Traffic Data Generation: A Comparative Analysis.

Sui, Y. and Zhou, M. and Zhou, M. and Han, S. and Zhang, D., 2024. Table Meets LLM: Can Large Language Models Understand Structured Table Data? A Benchmark and Empirical Study.

Hegselmann, S. and Buendia, A. and Lang, H. and Agrawal, M. and Jiang, X. and Sontag, D., 2023. TabLLM: Few-shot Classification of Tabular Data with Large Language Models.

Wei, J. and Wang, X. and Schuurmans, D. and others, 2022. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models.

Ding, N. and Qin, Y. and Yang, G. and Wei, F. and others, 2022. Delta Tuning: A Comprehensive Study of Parameter Efficient Methods for Pre-trained Language Models.